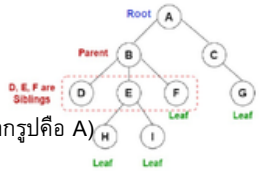
สรุปบทที่ 8: Tree (แผนภาพต้นไม้)

1. คุณสมบัติพื้นฐานของ Tree (ต้องจำ!)

Tree คือ Connected Graph (กราฟที่ทุกโหนดเชื่อมถึงกัน) ที่มีลักษณะพิเศษคือ:

- Acyclic: ไม่มีวัฏจักร (ไม่มีเส้นเชื่อมวนกลับมาที่เดิมจนเป็นวงปิด)
- No Loop / No Parallel Edges: ไม่มีเส้นวนเข้าหาตัวเอง และไม่มีเส้นเชื่อมซ้ำซ้อนระหว่าง 2 จุด
- Edges Count: ถ้ามีโหนด n ตัว จะต้องมีเส้นเชื่อม (Edges) ทั้งหมด n-1 เส้นเสมอ
- Empty Tree: คือ Tree ที่ไม่มีโหนดเลย (n = 0)



2. โครงสร้างและส่วนประกอบ (Terminology)

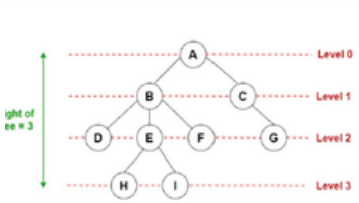
เวลาทำข้อสอบ มักจะถามหาชื่อเรียกหรือนับจำนวนเหล่านี้:

- Root (ราก): โหนดบนสุดโหนดเดียวที่ไม่มี Parent (จากรูปคือ A)
- Parent (พ่อแม่): โหนดที่มีลูกเชื่อมลงไปข้างล่าง
- Child (ลูก): โหนดที่ถูกเชื่อมมาจากด้านบน
- Siblings (พี่น้อง): โหนดที่มี Parent คนเดียวกัน (เช่น D, E, F เป็นพี่น้องกัน)
- Leaf (ใบ): โหนดที่ไม่มีลูก (External/Terminal Nodes) เช่น D, F, G, H, I
- Internal Node: โหนดที่ไม่ใช่ใบ (มีลูกอย่างน้อย 1 ตัว)

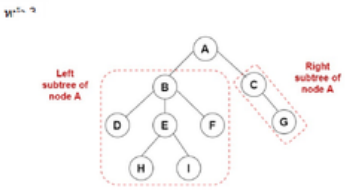
3. การวัดขนาดและระดับ (Measurement)

ส่วนนี้มักออกคำนวณเล็กๆ น้อยๆ ครับ:

- Degree (ดีกรี): * Degree of Node: จำนวนลูกของโหนดนั้น (เช่น deg(E) = 2, deg(B) = 3)
  - Degree of Tree: ดีกรีสูงสุดในต้นไม้นั้น (จากรูปคือ 3)
- Level & Height:
  - Level: เริ่มนับ Root เป็น Level 0 ถัดลงมาเป็น 1, 2, 3...
  - Height of Tree: ระดับ (Level) ที่สูงที่สุดของโหนดใน Tree นั้น (จากรูปในหน้า 3 ความสูงคือ 3)
- Depth (ความลึก): ระยะทางจาก Root ไปยังโหนดนั้นๆ (Root ความลึก = 0)
- Subtree (ต้นไม้ย่อย): กิ่งก้านที่แตกออกมา ซึ่งตัวมันเองก็มีโครงสร้างเป็น Tree



ภาพที่ 2 ส่วนสูงของทรี



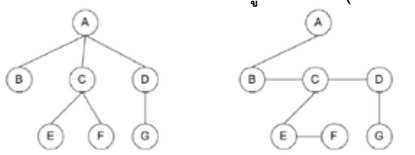
ภาพที่ 3 ต้นไม้ย่อย

ประเภทของ Tree (Types of Trees)

1. n-ary Tree (ทรีทั่วไป)

คือ Tree ที่โหนดแต่ละตัวมีลูกได้ไม่เกิน n ตัว (เช่น 3-ary tree คือมีลูกได้มากสุด 3 ตัว)

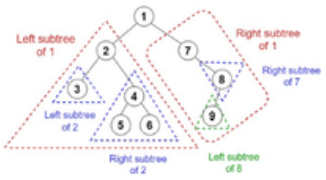
- Left-child Right-sibling: เป็นเทคนิคการเก็บข้อมูลทรีทั่วไปให้ประหยัด memory โดยโหนดหนึ่งจะเก็บ link แค่ 2 เส้น คือ ลูกคนแรก (ทางซ้าย) และ พี่น้องถัดไป (ทางขวา)

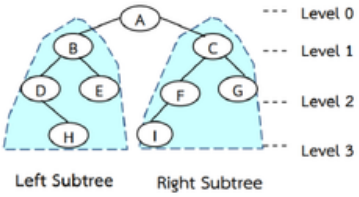


N-ary tree First child / Next sibling tree

2. Binary Tree (ต้นไม้ทวิภาค)

หัวใจสำคัญ: ทุกโหนดมีลูกได้ ไม่เกิน 2 ตัว (มีแค่ลูกซ้ายและลูกขวา) มักออกสอบเรื่อง "ชนิดย่อย" ของ Binary Tree ดังนี้:





ประเภท	คำนิยาม (จุดสังเกต)
Full Binary Tree	ทุกโหนด (ยกเว้นใบ) **ต้องมีลูก 2 ตัวเต็ม** (ไม่มีใครมีลูกคนเดียว)
Complete Binary Tree	เติมโหนดเต็มทุกชั้น **ยกเว้นชั้นสุดท้าย** โดยต้องเติมจาก **ซ้ายไปขวา** ห้ามแหว่ง
Perfect Binary Tree	สมบูรณ์แบบที่สุด! ทุกชั้นเต็มหมด และโหนดใบ (Leaf) ทุกตัวต้องอยู่ **ระดับเดียวกัน**
Skewed Binary Tree	ทรีที่เอียงไปด้านใดด้านหนึ่งเพียงด้านเดียว (เหมือน Linked List)

📏 สูตรคำนวณที่ควรจดลง A4 (สำหรับ Perfect Binary Tree)

หากโหนดมีความสูงเป็น h:

1. จำนวนโหนดทั้งหมด (n): n = 2^{h+1} - 1
2. จำนวนโหนดใบ (Leaf): มีค่าเท่ากับ (n+1) / 2
3. ความสูงของทรี (h): หาได้จาก log_2(n) (โดยประมาณ)

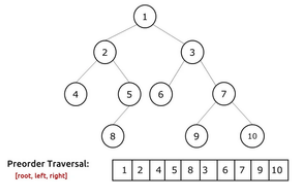
การดำเนินการ

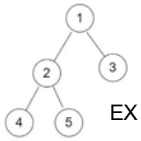
1. การท่องเข้าไปในทรี (Tree Traversal)

Pre-order Traversal (Prefix form)

ลำดับ: Root > Left Subtree > Right Subtree

ทริค: จดชื่อโหนดทันทีที่เจอครั้งแรก(จากด้านบนหรือซ้าย)



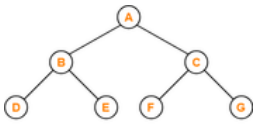


EX : 4 2 5 1 3

In-order Traversal (Infix form)

ลำดับ: Left Subtree > Root > Right Subtree

ทริค: ถ้าเป็น BST ผลลัพธ์ที่ได้จะเรียงลำดับจาก น้อยไปมากเสมอ



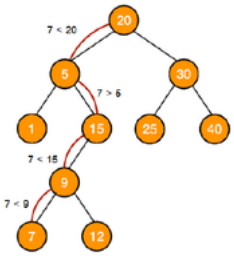
Post-order Traversal (Postfix form)

ลำดับ: Left Subtree > Right Subtree > Root

ทริค: Root จะเป็นตัวสุดท้ายของลำดับเสมอ

2. การดำเนินการใน Binary Search Tree (BST)

BST มีกฎสำคัญคือ: ซ้าย < พ่อแม่ < ขวา



1.1การค้นหาข้อมูล (Searching)

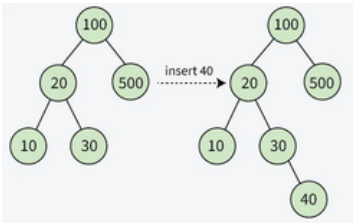
เปรียบเทียบค่าที่ต้องการกับ Root

1.ถ้าค่าน้อยกว่า ไปหาที่ต้นไม้ย่อยทาง ซ้าย

2.ถ้าค่ามากกว่า ไปหาที่ต้นไม้ย่อยทาง ขวา

3.ค่าที่เล็กที่สุด: ท่องไปทางซ้ายจนสุดโหนด

4.ค่าที่ใหญ่ที่สุด: ท่องไปทางขวาจนสุดโหนด



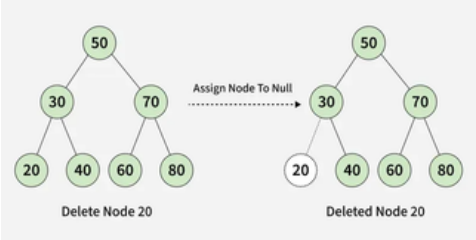
การเพิ่มข้อมูล (Insertion)

- ใช้วิธีการเดียวกับการค้นหา (Search) เปรียบเทียบค่าไปเรื่อยๆ จนเจอตำแหน่งที่เป็นโหนดใบ (Leaf) แล้วจึงเพิ่มเป็นลูกซ้ายหรือขวาตามค่าของมัน

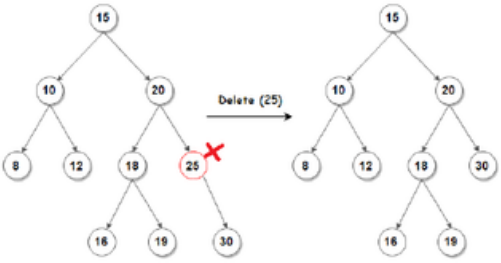
การลบข้อมูล (Deletion)

แบ่งเป็น 3 กรณีตามจำนวนลูกของโหนดที่จะลบ:

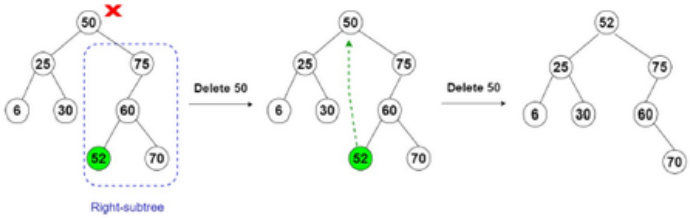
1. ลบโหนดใบ (Leaf node): ลบออกได้ทันที
2. มีลูก 1 ตัว: นำโหนดลูกขึ้นมาแทนที่โหนดที่ถูกลบ
3. มีลูก 2 ตัว: นำโหนดที่มี ค่าน้อยที่สุดในต้นไม้ย่อยทางขวา (In-order Successor) ขึ้นมาแทนที่ตำแหน่งที่ลบ



ลบโหนดใบ (Leaf node): ลบออกได้ทันที



มีลูก 1 ตัว: นำโหนดลูกขึ้นมาแทนที่โหนดที่ถูกลบ



มีลูก 2 ตัว: นำโหนดที่มี ค่าน้อยที่สุดในต้นไม้ย่อยทางขวา (In-order Successor) ขึ้นมาแทนที่ตำแหน่งที่ลบ

ตัวอย่างการลบ (จากเอกสารหน้า 35-36)

- **โจทย์:** ต้องการลบโหนด **50** ซึ่งมีลูก 2 ตัว (คือ 25 และ 75) +2
- **ขั้นตอน:** * ดูในฝั่งขวา (Right-subtree) ของ 50 ซึ่งมีโหนด 75, 60, 52, 70 +3
  - หาค่าที่ **น้อยที่สุด** ในกลุ่มนั้น คือ **52** +2
  - นำ **52** ขึ้นไปแทนที่ **50** +1
  - ผลลัพธ์คือ 50 หายไป และมี 52 เป็นรากแทน โดยที่โครงสร้างทรียังถูกต้องตามกฎของ BST +1

สรุป Graph (ชุดที่ 1)

1. นิยามสำคัญ (Definitions)

- Graph $G = \{V, E\}$: ประกอบด้วย V (Vertices/Nodes) คือจุดยอด และ E (Edges) คือเส้นเชื่อม
- Adjacent (จุดประชิด): จุดที่มีเส้นเชื่อมถึงกันโดยตรง
- Path (เส้นทาง): ลำดับของจุดที่เชื่อมต่อกัน
  - Simple Path: เส้นทางที่ "ไม่เดินซ้ำจุดเดิม"
- Degree (ดีกรี): จำนวนเส้นที่เชื่อมกับจุดนั้น
  - In-degree: จำนวนเส้นที่ "ชี้เข้า"
  - Out-degree: จำนวนเส้นที่ "ชี้ออก"
  - Total Degree: $In + Out$

2. ประเภทของกราฟ (Graph Types)

ส่วนนี้มักออกสอบแบบให้รูปมาแล้วถามว่าเป็นกราฟประเภทไหน

1. แบ่งตามทิศทาง (Direction)

- กราฟไม่ระบุทิศทาง (Undirected graph): ทุกเส้นเชื่อมในกราฟไม่มีหัวลูกศรกำกับ บอกเพียงว่าจุดสองจุดเชื่อมต่อกัน
  - ตัวอย่าง: Facebook Friends Graph เมื่อรับเป็นเพื่อนกัน ทั้งคู่จะเชื่อมถึงกันโดยอัตโนมัติ
- กราฟระบุทิศทาง (Directed graph): ทุกเส้นเชื่อมมีการระบุทิศทาง (มีหัวลูกศร)
  - ตัวอย่าง: Instagram Followers Graph การที่เราติดตามใครคนหนึ่ง ไม่ได้หมายความว่าเขาจะติดตามเรากลับเสมอไป
  - ดีกรี (Degree) ของกราฟประเภทนี้จะแบ่งเป็น In-degree (เส้นชี้เข้า) และ Out-degree (เส้นชี้ออก) โดยดีกรีรวมหาได้จาก Indeg + Outdeg

2. แบ่งตามการมีวัฏจักร (Cycle)

- Cyclic graph: กราฟที่มีเส้นทาง (Path) ซึ่งจุดเริ่มต้นและจุดปลายทางเป็นจุดเดียวกันอย่างน้อยหนึ่งวงจร
- Acyclic graph: กราฟที่ไม่มีวงจรปิดหรือวัฏจักรเลย
  - กราฟระบุทิศทางที่ไม่มีวัฏจักรจะเรียกว่า DAG (Directed Acyclic Graph)

3. แบ่งตามการมีเส้นขนาน (Parallel) และลูป (Loop)

- กราฟทั่วไป (Simple graph): แต่ละคู่ของจุดมีเส้นเชื่อมเพียงเส้นเดียว และไม่มีลูป
- มัลติกราฟ (Multi graph): บางคู่ของจุดมีเส้นเชื่อมได้มากกว่าหนึ่งเส้น (Parallel edges) แต่ยังไม่มีลูป
- ซูโดกราฟ (Pseudo graph): มีทั้งเส้นเชื่อมขนานและมีลูป (เส้นที่เชื่อมจากจุดกลับมายังตัวเอง)

4. แบ่งตามจำนวนดีกรีของโหนด

- Discrete graph (Null graph): กราฟที่มีแต่จุดแต่ไม่มีเส้นเชื่อมเลย (ดีกรีเป็น 0 ทุกโหนด)
- Linear graph (Path graph): กราฟที่เชื่อมต่อกันเป็นสาย โดยโหนดหัวและท้ายมีดีกรี 1 ส่วนโหนดระหว่างกลางมีดีกรี 2
- Regular graph: กราฟที่ทุกโหนดมีดีกรีเท่ากันหมด (เช่น 2-Regular คือทุกโหนดมีดีกรี 2)
- Complete graph (K_n): กราฟที่ทุกโหนดเชื่อมต่อกับโหนดอื่นๆ ทุกโหนดที่เหลือ

5. กราฟมีน้ำหนัก (Weighted Graph)

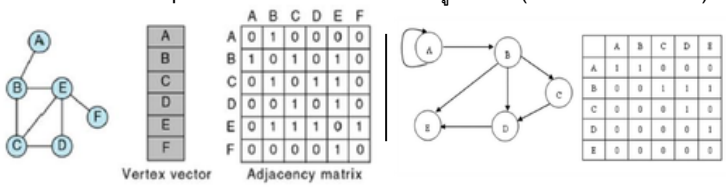
- คือกราฟที่แต่ละเส้นเชื่อมมีค่าตัวเลขกำกับ ซึ่งอาจแทนด้วยระยะทาง, เวลา, ค่าใช้จ่าย หรือความเร็ว โดยปกติค่าน้ำหนักมักไม่เป็นค่าลบ แต่อาจเป็นศูนย์ได้ในบางกรณี (เช่น ปริมาณการจราจรบนถนนที่ไม่มีรถวิ่งเลย)

การนำเสนอกราฟ (Graph Representation)

1. Adjacency Matrix (เมทริกซ์ประชิด)

เป็นการใช้ ตาราง 2 มิติ (Array 2D) ขนาด n * n (เมื่อ n คือจำนวนโหนด)

- วิธีการลงข้อมูล: * ถ้าโหนด i เชื่อมกับโหนด j ให้ใส่ค่า 1
  - ถ้าไม่เชื่อมกัน ให้ใส่ค่า 0
  - ถ้าเป็น Weighted Graph (กราฟมีน้ำหนัก) ให้ใส่ "ค่าน้ำหนัก" แทนเลข 1 และใส่ ∞ หรือ 0 แทนจุดที่ไม่มีเส้นเชื่อม
- ข้อสังเกต:
  - Undirected Graph: ตารางจะเป็นรูปกระจกเงา (Symmetric) แนวทแยงมุม
  - Directed Graph: ตารางอาจไม่สมมาตร ต้องดูทิศทาง (จากแถว ไปยัง หลัก)



(a) Adjacency matrix for non-directed graph

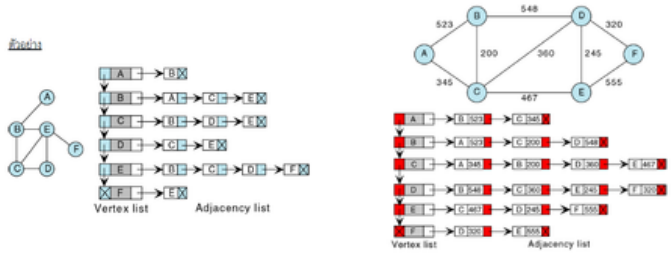
2. Adjacency List (รายการประชิด)

เป็นการใช้ Linked List หรือ Array ของ List เก็บเฉพาะโหนดที่มีเส้นเชื่อมถึงกันจริงๆ

- วิธีการลงข้อมูล: * สร้างหัวแถวตามจำนวนโหนด
  - โหนดไหนเชื่อมไปหาใคร ให้เขียนต่อท้ายเป็นทอดๆ (เช่น A-> B -> C)
- ข้อสังเกต:
  - ถ้าเป็น Weighted Graph ใน List จะต้องเก็บเป็นคู่ลำดับเช่น (จุดปลายทาง, น้ำหนัก)

Trick การจำ: > * Matrix = ตาราง: ดูง่ายแต่เปลืองที่ (เหมือนจองที่นั่งเผื่อไว้ทุกจุด)

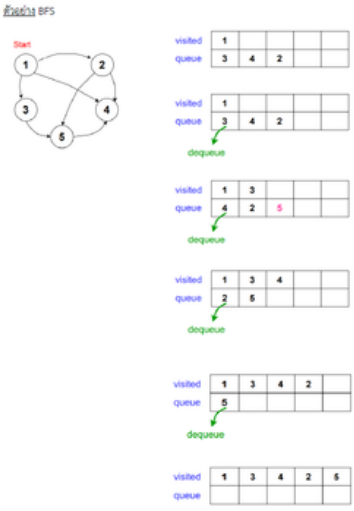
- List = รายชื่อ: ประหยัดที่แต่ค้นหาช้า (เหมือนจดเฉพาะชื่อเพื่อนที่มาจริง)



การท่องไปในกราฟ (Graph Traversal)

1. Breadth First Search (BFS) - การค้นหาตามแนวขวาง

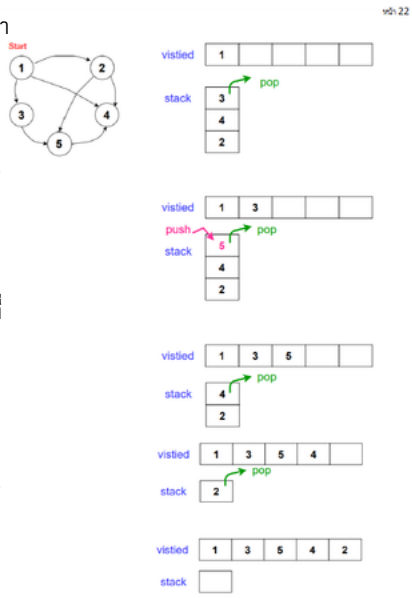
เน้นกระจายออกไปเยี่ยมโหนดเพื่อนบ้านให้ครบก่อน แล้วค่อยขยับไปชั้นถัดไป (เหมือนวงน้ำที่กระจายออกจากจุดศูนย์กลาง)



- โครงสร้างข้อมูลที่ใช้: Queue (คิว) - FIFO
- หลักการทำงาน:
  a. เริ่มที่โหนดเริ่มต้น ใส่ลงใน Queue และทำเครื่องหมายว่า "เยี่ยมแล้ว" (Visited)
  b. ดึงโหนดออกจาก Queue แล้วไปดูว่ามันเชื่อมกับโหนดไหนบ้างที่ยังไม่เคยไป
  c. ใส่โหนดเพื่อนบ้านเหล่านั้นลงใน Queue และทำเครื่องหมาย Visited
  d. ทำซ้ำจนกว่า Queue จะว่าง
- จุดเด่น: ใช้หา "ระยะทางที่สั้นที่สุด" (Shortest Path) ในกราฟที่ไม่มีน้ำหนักได้ดี

2. Depth First Search (DFS) - การค้นหาตามแนวลึก

เน้นเดินหน้าไปให้สุดสายทางก่อน ถ้าตันแล้วค่อยย้อนกลับมา (Backtrack) เลือกทางแยกถัดไป



- โครงสร้างข้อมูลที่ใช้: Stack (สแต็ก) - LIFO (หรือใช้ Recursion)
- หลักการทำงาน:
  a. เริ่มที่โหนดเริ่มต้น ใส่ลงใน Stack และทำเครื่องหมาย Visited
  b. เลือกโหนดเพื่อนบ้านมา 1 โหนดที่ยังไม่เคยไป แล้วลุยลึกเข้าไปต่อทันที
  c. หากไปจนสุดทาง (ไม่มีทางไปต่อ) ให้ย้อนกลับ (Backtrack) มาที่โหนดก่อนหน้าเพื่อหาทางแยกอื่น
  d. ทำซ้ำจนกว่าจะกลับมาที่จุดเริ่มต้นและไม่มีทางไปต่อแล้ว
- จุดเด่น: ใช้ตรวจสอบการเกิด Cycle (วัฏจักร) ในกราฟ และใช้พื้นที่หน่วยความจำน้อยกว่าในบางกรณี